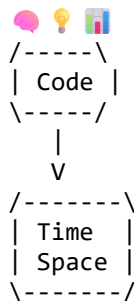


Time Complexity: Essential Study Notes

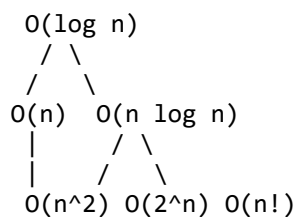
Understanding Algorithm Performance 🚀



Introduction to Time Complexity

- **What is Time Complexity?**
 - The *time taken by an algorithm as a function of the length of input*.
 - It describes how an algorithm's performance changes as the input size increases.
- **Why is it Important?** 🤔
 - Crucial for interviews and competitive programming.
 - Helps *compare two algorithms* to determine which one is more efficient.
 - Avoids issues with performance measurement variations due to hardware or execution environment.
- **Example: Sum from 1 to N**
 - Method 1: Direct formula $N(N+1)/2$.
 - Method 2: Using a `for` loop from 1 to N.
 - Both are valid, but *time complexity helps determine which is better*.
- **Key Concepts Covered:**
 - Big O Notation 📈
 - Time Complexity of Recursive Functions (Tree Method) 🌳
 - Online Judges and Constraints (10^8 Operations Rule) 🚦

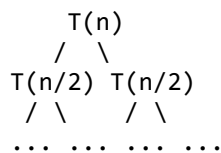
Big O Notation



- **Definition:**
 - A mathematical notation that *quantifies the time complexity* of an algorithm.

- It describes the **upper bound** or **worst-case scenario** of an algorithm's running time.
 - "This algorithm will take **at most** this much time."
- Worst-Case Complexity:**
 - Big O Notation typically represents the worst-case scenario.
 - Example (Linear Search):* The worst case is when the element is not present in the array, or it is the last element. The algorithm has to traverse the entire array.
- Rules for Determining Big O:**
 - Focus on the Largest Power (Dominant Term):** Ignore smaller powers of N.
 - Ignore Constants:** Ignore constant factors associated with the largest power.
- Examples of Big O Calculation:**
 - $3n^2 + 5n \rightarrow O(n^2)$ (Ignoring $5n$ and 3)
 - $n + 100 \log n \rightarrow O(n)$ (n grows faster than $100 \log n$)
 - $3n^3 + 4n^5 \rightarrow O(n^5)$ (Ignoring $3n^3$ and 4)
 - $10000 \rightarrow O(1)$ (Constant time, not dependent on input n)
 - $n/4, n + 100/4, 5n + 3 \log n \rightarrow O(n)$
 - $n^2 / 1000 \rightarrow O(n^2)$
- Other Notations (Briefly):**
 - Theta (Θ) Notation:** Describes the **exact bound** (tight bound) of an algorithm.
 - Omega (Ω) Notation:** Describes the **lower bound** (best-case scenario) of an algorithm.
 - Big O is most commonly used in computer science because it provides a safe upper bound.*

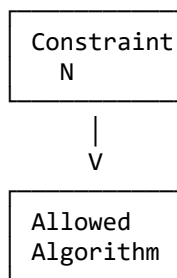
Time Complexity of Recursive Functions (Tree Method)



- Challenge:** Finding the Big O of functions that call themselves.
- Method:** Tree Method 🌲
- Steps:**
 - Convert to T-function (Recurrence Relation):**
 - Express the total time $T(n)$ in terms of recursive calls and non-recursive work.
 - Example:* A function $\text{fun}(n)$ that calls $\text{fun}(n/2)$ twice and has a *'for'* loop running n times.
 - $T(n) = 2T(n/2) + O(n)$ (or $n \cdot c$ for non-recursive work)
 - Visualize as a Tree:**
 - The root represents the non-recursive work at the initial call $T(n)$.
 - Children nodes represent the recursive calls.
 - Expand the Tree:**
 - Continue expanding until a pattern emerges for the work done at each level.

- *Example:*
 - Level 0: nc operations
 - Level 1: $(n/2)*c + (n/2)*c = nc$ operations
 - Level 2: $(n/4)*c * 4 = nc$ operations
 - *Pattern:* nc operations performed at every step/level.
- 4. **Determine Tree Height:**
 - Identify how many levels the recursion goes.
 - If n is halved at each step ($n, n/2, n/4, \dots$ until $n=0$), the height is **$\log n$** .
- 5. **Calculate Total Operations:**
 - Total operations = (operations per level) * (number of levels)
 - *Example:* $nc * \log n$
- 6. **Apply Big O Rules:**
 - Remove constants.
 - *Example:* $O(n \log n)$
- **Practice Questions:**
 - $T(n) = T(n/4) + T(n/2) + c$
 - $T(n) = 2T(n-1) + c$

Online Judges and Constraints (10^8 Operations Rule)



- **Purpose:** To quickly determine which algorithm to use based on input constraints to avoid "Time Limit Exceeded" (TLE) errors.
- **The 10^8 Operations Rule:**
 - Online servers typically allow a code to perform approximately 10^8 operations* within the given time limit (e.g., 1 second).
 - If your code exceeds this, you'll get a TLE.
- **Algorithm Selection Based on N:**
 - If N is 10-11: Algorithms up to $O(N!)$ or $O(N^6)$ might pass.
 - If N is 15-18: Algorithms with complexity like $O(2^N * N)$ might pass.
 - If N is 100: Algorithms up to $O(N^4)$ might pass.
 - If N is 400: Algorithms up to $O(N^3)$ might pass.
 - If N is 10^5 : Algorithms like $O(N \log N)$ (e.g., sorting) or $O(N)$ might pass.
 - If N is 10^8 : Algorithms must be $O(N)$ or $O(\log N)$.
- **Benefit:** Helps in competitive programming to quickly choose an appropriate algorithm and avoid TLE.

Good luck with your studies! Keep practicing time complexity questions. 📖